

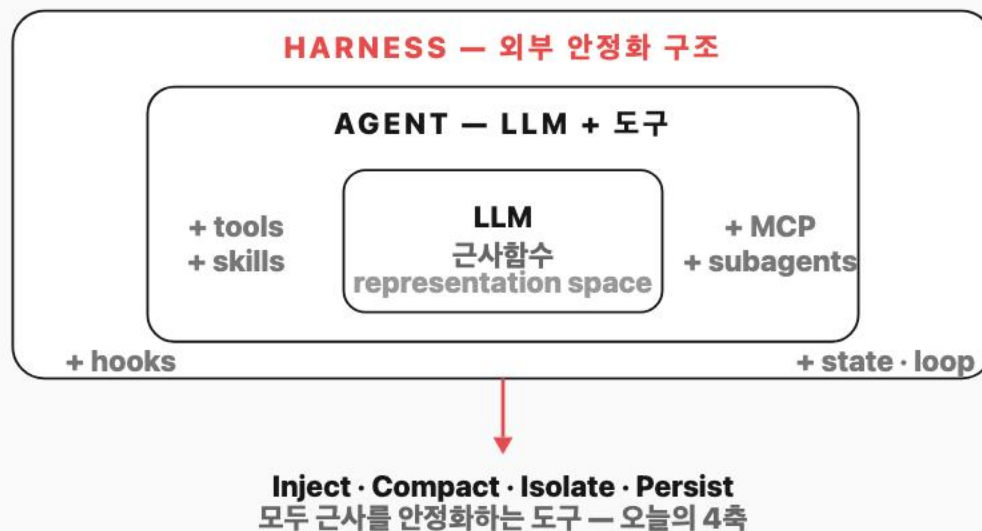
PYCON BUSAN 2026

나만의 Claude Code Harness

프롬프트를 반복 가능한 작업 시스템으로 바꾸는 법

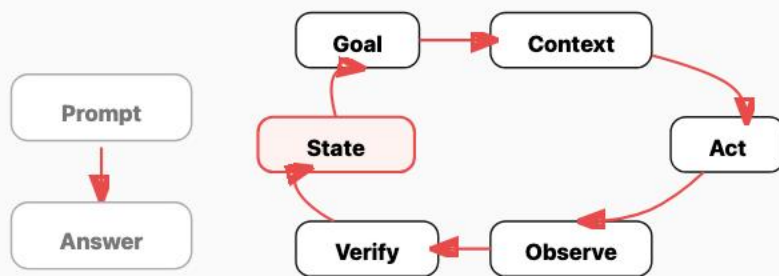
오늘의 한 줄 — LLM · Agent · Harness

LLM은 자연어를 representation space에서 근사하는 함수입니다. Agent는 거기에 도구를 더한 것, Harness는 그 근사가 흐트러지지 않게 외부에서 잡아주는 구조입니다.

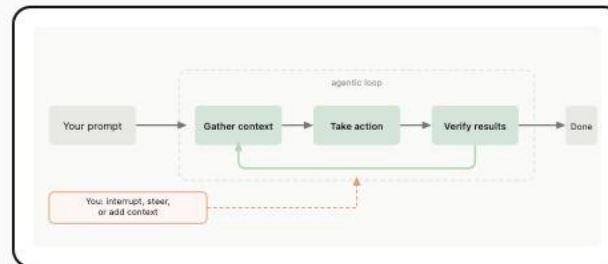


작업 단위가 바뀌었다

예전에는 “질문→답”이 작업 단위였습니다. 이제는 목표가 행동·관찰·검증·상태로 반복됩니다 — Claude Code 공식 문서가 같은 그림을 씁니다.



CLAUDE CODE 공식 — 에이전트 루프



gather context → take action → verify, 작업 끝까지 반복.
code.claude.com/docs/ko/how-claude-code-works

Claude Code가 똑똑해도 헛도는 이유

문제는 모델 지능만이 아닙니다. 실패와 관찰이 다음 loop의 좋은 상태로 바뀌지 않으면 계속 헛돕니다.

실패

테스트 로그가 대화에 묻힘

망각

어제의 결정이 오늘 사라짐

오염

실행·검증·외부 문서가 섞임

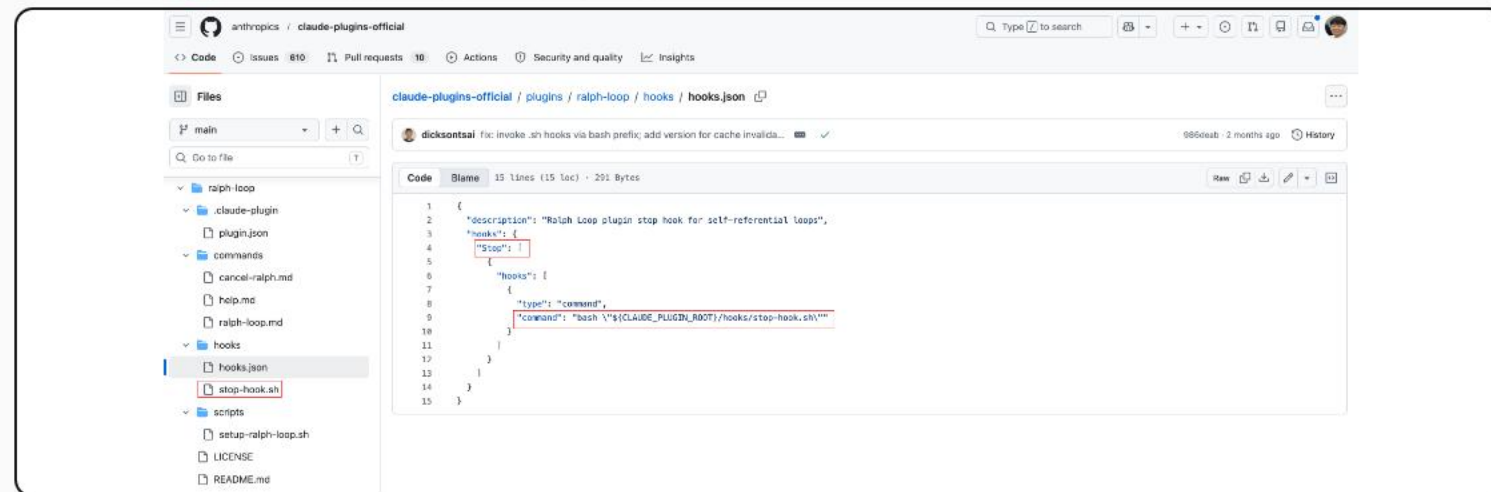
Ralph Loop: 가장 작은 반복 구조

반복 자체는 단순합니다. 중요한 건 progress가 transcript가 아니라 repo/files/git에 산다는 점입니다 — Anthropic도 같은 패턴을 공식 plugin으로 받아들였습니다.

```
while ;; do cat PROMPT.md | claude-code ; done
```

cultural anchor — production safety recipe 아님

ANTHROPIC 공식 RALPH-LOOP PLUGIN — STOP HOOK



github.com/anthropics/claude-plugins-official/blob/main/plugins/ralph-loop/hooks/hooks.json

stop-hook.sh: 작업이 끝나야 종료

Stop hook이 prompt를 다시 Claude에 피드백합니다. 모델이 “다 됐다”고 판단해도 completion promise 통과 전엔 loop가 종료되지 않습니다.

RALPH-LOOP/HOOKS/STOP-HOOK.SH — 핵심 부분

```

152 echo " PROMPT: new prompt text found" >>2
153 echo "" >>2
154 echo " This usually means:" >>2
155 echo "   • State file was manually edited" >>2
156 echo "   • File was corrupted during writing" >>2
157 echo "" >>2
158 echo " Ralph loop is stopping. Run /ralph-loop again to start fresh." >>2
159 rm "$RALPH_STATE_FILE"
160 exit 0
161
162 # Update iteration in frontmatter (portable across macOS and Linux)
163 # Create temp file, then atomically replace
164 TEMP_FILE="${RALPH_STATE_FILE}.tmp.$$"
165 sed "s/~iteration: .*/iteration: $NEXT_ITERATION/" "$RALPH_STATE_FILE" > "$TEMP_FILE"
166 mv "$TEMP_FILE" "$RALPH_STATE_FILE"
167
168 # Build system message with iteration count and completion promise info
169 if [[ "$COMPLETION_PROMISE" != "null" ]] 66 [[ -n "$COMPLETION_PROMISE" ]]; then
170   SYSTEM_MSG="@ Ralph iteration $NEXT_ITERATION | To stop: output <promise>$COMPLETION_PROMISE</promise> (ONLY when statement is TRUE - do not lie to exit!)"
171 else
172   SYSTEM_MSG="@ Ralph iteration $NEXT_ITERATION | No completion promise set - loop runs infinitely"
173 fi
174
175 # Output JSON to block the stop and feed prompt back
176 # The "reason" field contains the prompt that will be sent back to Claude
177 jq -n \
178   --arg prompt "$PROMPT_TEXT" \
179   --arg msg "$SYSTEM_MSG" \
180   '{
181     "decision": "block",
182     "reason": $prompt,
183     "systemMessage": $msg
184   }'
185
186 # Exit 0 for successful hook execution
187 exit 0

```

github.com/anthropics/claude-plugins-official/blob/main/plugins/ralph-loop/hooks/stop-hook.sh

Karpathy Loop: 실험 loop 설계

가설을 세우고, 바꾸고, 짧게 실험하고, metric이 좋아지면 keep — 아니면 revert합니다.

hypothesis →

change →

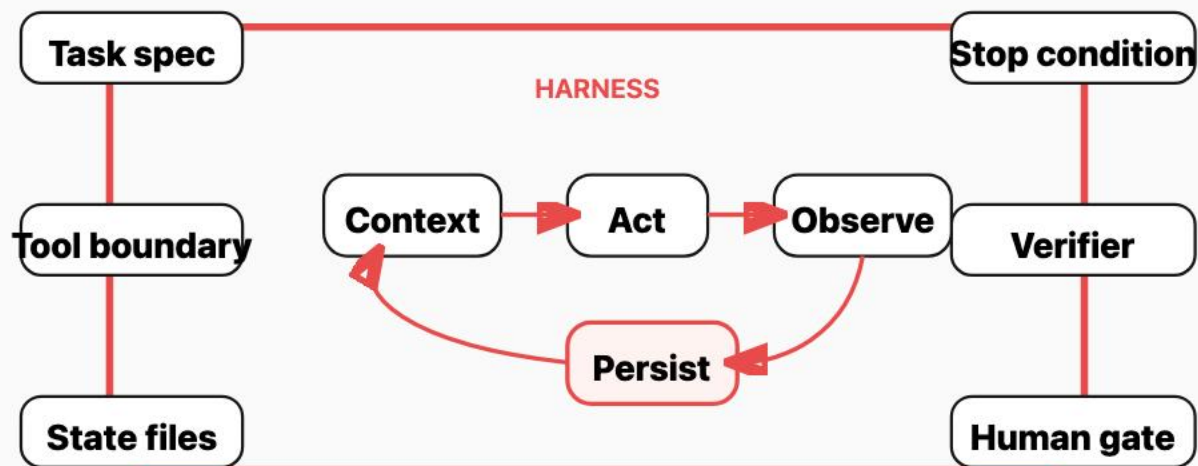
5min run →

metric →

keep / revert

오늘의 전환

Harness란 이 반복이 안전하게 누적되도록 만드는 바깥 구조입니다.



Harness는 설정 묶음이 아니다

Claude Code 공식 문서가 같은 단어를 씁니다 — 모델을 둘러싼 **agent harness**가 Claude Code의 정의입니다.

CLAUDE CODE 공식 정의

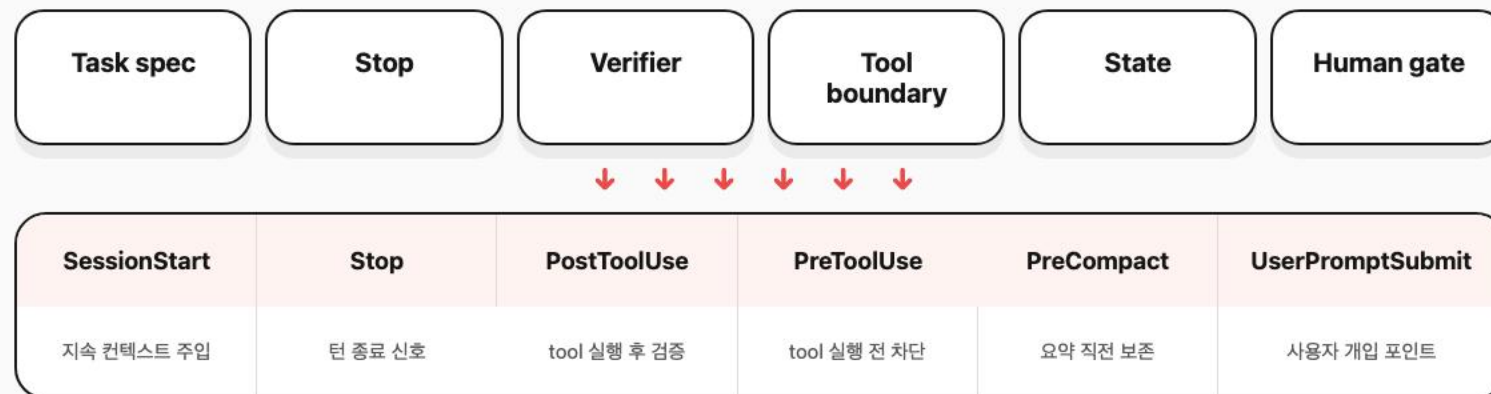
"Claude Code는 Claude 주변의 에이전트 하네스 역할을 합니다."

언어 모델을 능력 있는 코딩 에이전트로 변환하는 도구-컨텍스트 관리 실행 환경.

code.claude.com/docs/ko/how-claude-code-works

Harness 6 부품을 hook으로 만진다

추상 부품은 settings.json의 hook event로 내려옵니다. 손으로 만질 수 있어야 진짜 harness입니다.



8 hook event 중 핵심 6개 — code.claude.com/docs/en/hooks

끝나는 조건 없이는 **loop**가 아니다

DONE, tests, metric, human checkpoint가 있어야 반복이 “자동 삽질”이 아니라 설계된 작업이 됩니다.

DONE

tests pass

metric improves

human review

State는 context 밖에 산다

Context window는 휘발됩니다. `.claude/`의 파일들이 다음 iteration을 이어받습니다 — disposable과 durable의 분리가 harness의 핵심입니다.

context window

매 세션마다 휘발
disposable

`.claude/ durable substrate`

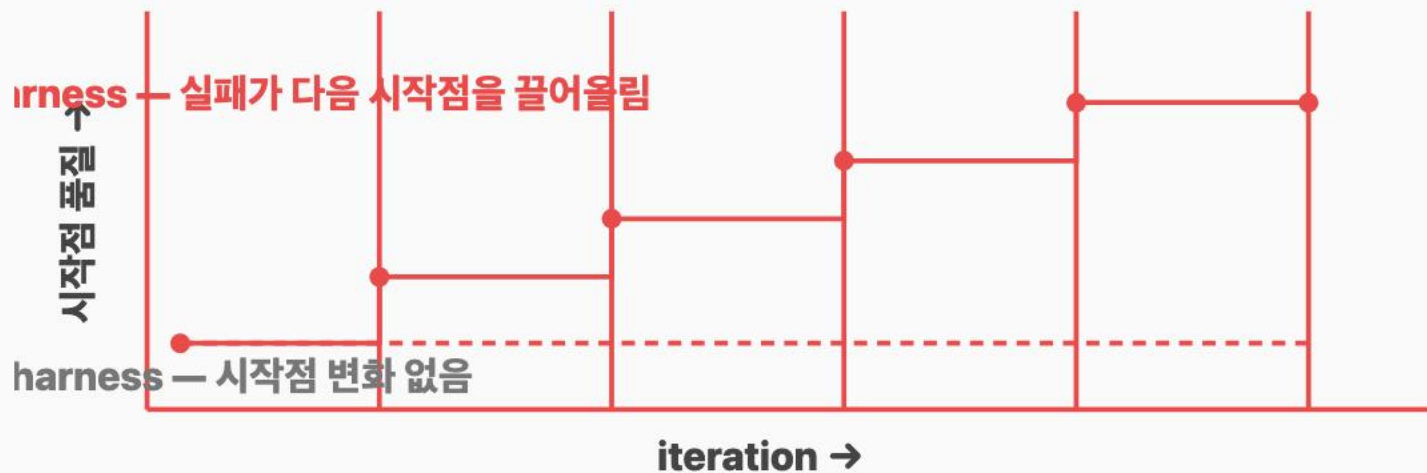
```
your-project/
└─ .claude/
    ├── CLAUDE.md 매 세션 자동 로드
    ├── settings.json 권한·hook 설정
    ├── agents/ subagent 정의
    ├── skills/ 재사용 워크플로우
    ├── commands/ /명령
    └─ hooks/ 스크립트
```

code.claude.com/docs/ko/claude-directory

좋은 harness의 기준

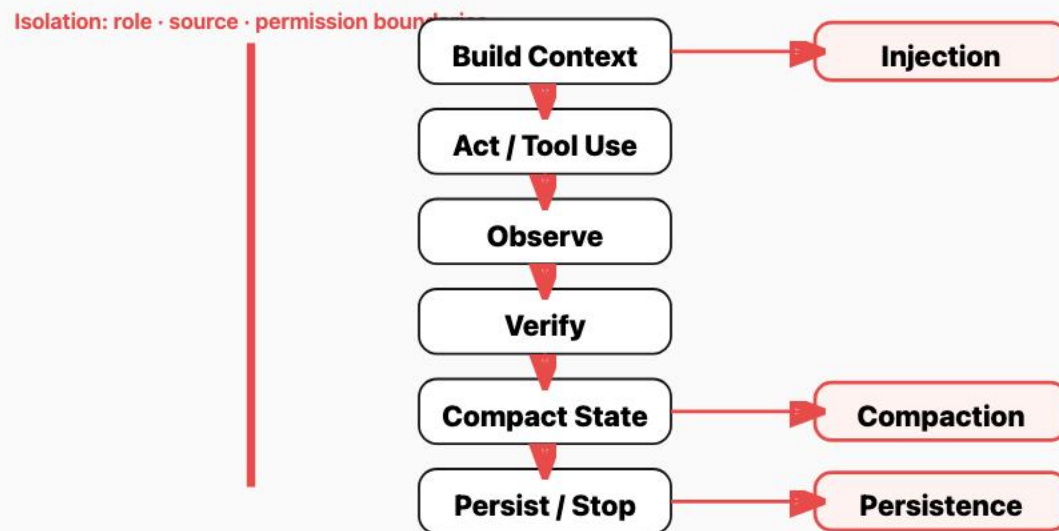
첫 답이 완벽한가보다, 실패 후 다음 시작점이 더 좋아졌는가가 기준입니다. 실패 자체가 자산이 됩니다.

반복 ITERATION의 시작점 품질



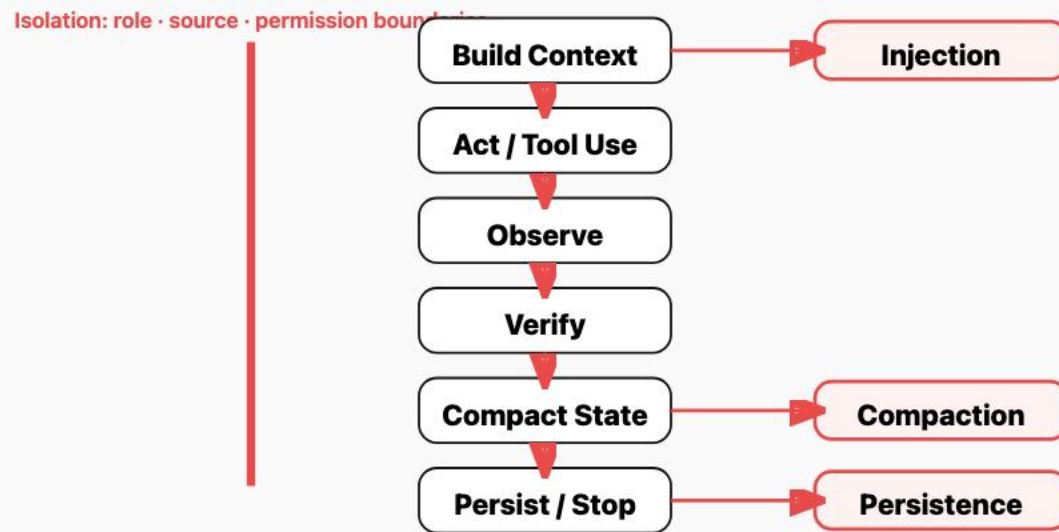
Context engineering = lifecycle design

Context는 한 번 넣고 끝나는 prompt가 아니라, 매 loop 갱신되는 working set입니다.



4축은 loop 안의 decision이다

Injection · Compaction · Isolation · Persistence는 네 챕터가 아니라 매 iteration의 설계 결정입니다.



Injection — 다음 행동을 바꿀 정보만

“알아야 할 것”이 아니라 다음 action을 바꾸는 evidence만 주입. context window가 커도 실효 출력은 60–70% 부근에서 수렴합니다.

goal

이번 loop의 목표

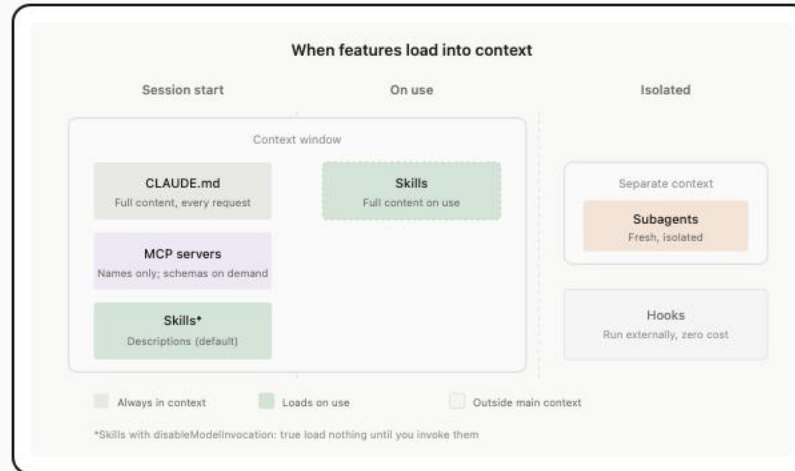
evidence

failing test · diff · 상태

constraint

금지 조건 · 취향

컨텍스트 로딩 — 공식 도식



유효용량 60–70%

window가 커도 신뢰 가능 출력은 일정 비율에서 수렴 — Chroma 2025

Injection 실패 모드

많이 넣었는데 행동이 바뀌지 않으면 noise입니다. MCP 서버를 붙일수록 tool schema가 budget을 잡아먹습니다.

많이 넣음
noise ↑

행동을 바꿈
signal ↑

TOOL SCHEMA — 등록 도구가 늘수록 평창

기본 도구 ~3K

+ MCP 5 servers ~8K (2~3x)

free

tool schema와 상시 지시는 매 요청마다 카운트 — Anthropic 2025

Compaction — 로그를 state로

요약은 줄이는 일이 아닙니다. transcript와 tool log를 다음 판단 가능한 상태로 재작성하는 일입니다.

CONTEXT WINDOW 200K — 채워지는 모습



compaction은 raw log를 다음 loop가 읽을 **state**로 재작성합니다 — decisions · failed attempts · open issues · next action.

시뮬레이션 — code.claude.com/docs/en/context-window

Tool output⁰ | context를 먹는다

Agent 궤적의 큰 비중은 observation stream입니다. 처리 끝난 출력은 state로 바꾸고 내려놓아야 합니다.

CODING AGENT 한 **TASK** 궤적의 토큰 점유율

tool output / observations $\frac{2}{3}$ – $\frac{7}{8}$

prompt + 응답

Coding agent의 한 task 궤적 토큰을 분석한 결과 약 $\frac{2}{3}$ – $\frac{7}{8}$ 이 **tool** 출력. 처리 끝난 observation은 끌고 가지 말고 **state**로 바꾸고 내려놓습니다.

Xiao et al., FSE 2026 (코딩 에이전트 토큰 궤적 분석)

Isolation — 역할과 권한 분리

Planner, executor, verifier, memory-writer는 서로 다른 context와 권한을 가져야 합니다.

planner

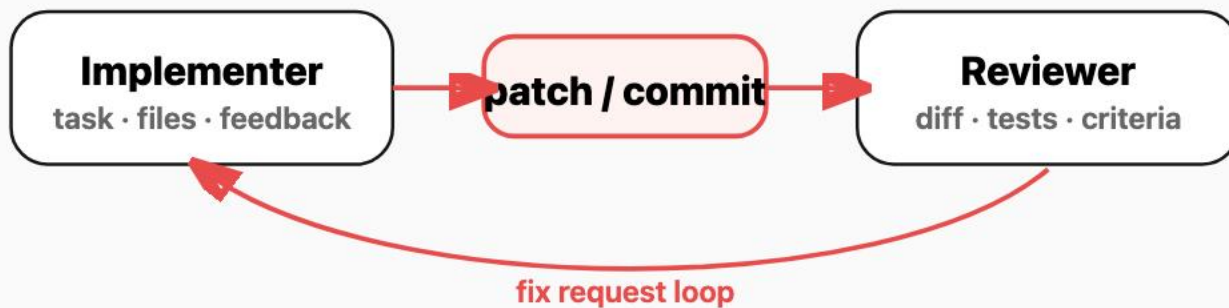
executor

verifier

memory writer

Verifier는 독립 context가 필요하다

만든 agent가 자기 결과를 검증하면 느슨해집니다. 검증은 별도 맥락에서 해야 합니다.



Persistence — 다시 발견하지 않게

기억을 많이 쌓는 게 아니라, 다음 agent가 다시 발견하지 않아도 될 state만 남깁니다.
memory 시스템의 가치는 측정 가능합니다.

git

변경의 감 식별자

progress.md

다음 loop의 상태

skill

반복 절차의 승격

LETTA — long-term memory ROI**+74%**정확도 상승
(Letta blog 2025)**ZEP — temporal knowledge graph****-90%**latency 감소
(arXiv:2501.13956)

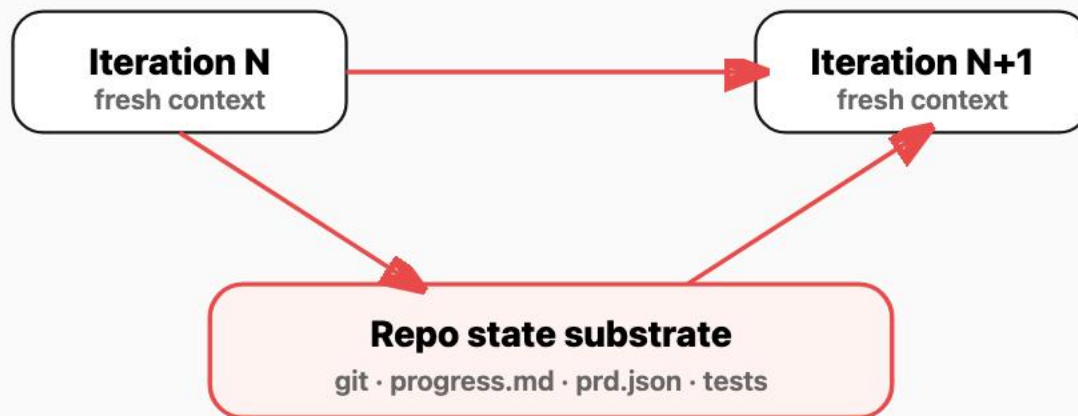
4축 한장 요약

좋은 loop는 넣고, 압축하고, 분리하고, 남깁니다. 네 축은 모두 다음 loop의 시작점을 바꿉니다.

Injection	Compaction	Isolation	Persistence
action-changing evidence	logs → state	roles / permissions	durable substrate

Pattern 1: Ralph-style coding loop

검증 가능한 coding task에서 Ralph-style loop는 “진행 상태를 repo에 남기며” 반복합니다.



Ralph의 핵심 파일

PROMPT.md는 의도, progress.md는 기억, tests는 verifier, git은 시간축입니다.

PROMPT.md

progress.md

guardrails.md

requirements.json

tests/

git commits

PROMPT.md

```
Goal: ship harness
Stop: tests pass
Verify: ./verify.sh
Don't: edit settings/
```

progress.md

```
[x] PROMPT 작성
[x] verifier 셋업
[ ] CI 통합 (다음)
open: rate_limit
```

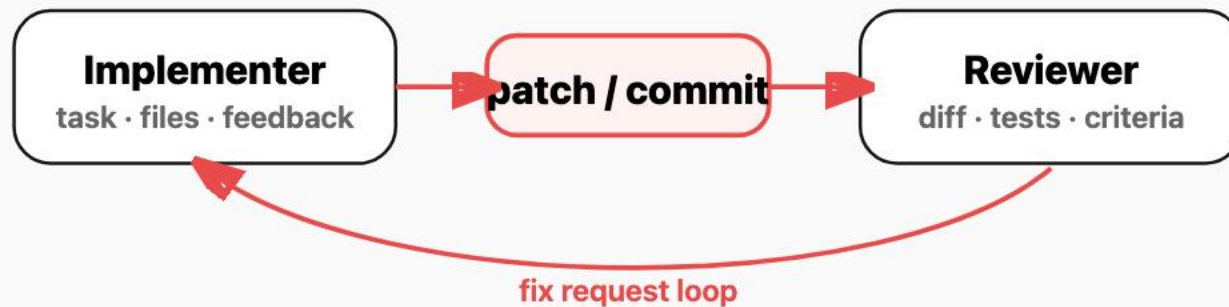
requirements.json

```
{
  "stories": [
    {"id": "S1", "pass": true},
    {"id": "S2", "pass": false}
  ]
}
```

의도 · 기억 · 검증 — repo가 substrate

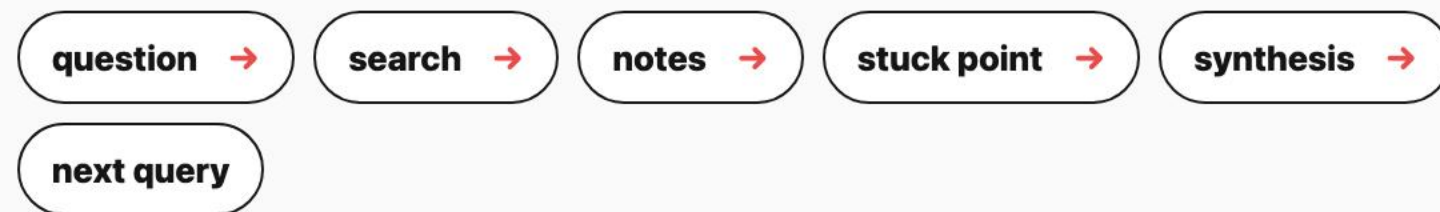
Pattern 2: Implementer / Reviewer loop

구현과 검증을 분리하면 자기합리화를 줄이고, patch loop가 선명해집니다.



Pattern 3: Research loop

Research도 한 번의 검색이 아니라 질문·검색·노트·synthesis·다음 질문의 반복입니다.



Pattern 4: Pretzel/Hermes loop

지금 이 텍스트 개편도 Google Tasks queue, project files, git commit, browser check, Slack report로 도는 harness입니다.



사례 비교 — 같은 4축, 다른 점유

각 pattern은 4축을 서로 다르게 점유합니다. 정답은 밀도가 아니라 loop의 모양입니다.

	Inject	Compact	Isolate	Persist
Ralph	●	●●	●	●●●
Review	●●	●	●●●	●
Research	●	●●●	●	●●
Pretzel	●●	●	●●	●●●

● 점유 강도 — 같은 4축이지만 pattern마다 무게중심이 다르게 놓임

나의 반복 작업 하나 고르기

모든 일을 자동화하지 마세요. 반복되는 작업 하나를 고르고, loop가 돌아갈 조건부터 정합니다.

반복됨

자주 돌아오는 작업

검증 가능

pass/fail이 있음

상태 누적

다음번에 이어받음

첫 harness는 세 파일이면 충분하다

처음부터 거대한 시스템이 필요하지 않습니다. task.md, progress.md, verify.sh부터 시작하세요.

task.md

progress.md

verify.sh

task.md

무엇을 끝낼지

Q3 OKR 자동 리포트
입력: GA · Stripe
출력: weekly.md

progress.md

week 1

[x] schema 매핑
[] retention 쿼리
open: rate_limit

verify.sh

#!/bin/bash
python check.py \
weekly.md \
|| exit 1

의도 · 기억 · 검증 — 세 파일이 loop를 잡아준다

언제 sub-agent를 쓰나

큰 탐색, 독립 검증, 다른 역할이 필요할 때만 격리합니다. 단순 작업은 비용만 늘 수 있습니다.

CLAUDE 공식 — 트리거 → 추가

트리거	추가
같은 프롬프트 두 번 입력	skill
출력이 대화 범람	subagent
두 번째 repo 동일 설정	plugin
매번 자동 실행 원함	hook

code.claude.com/docs/ko/features-overview

큰 탐색? → sub-agent

검증 독립성? → reviewer

작업 충돌? → worktree

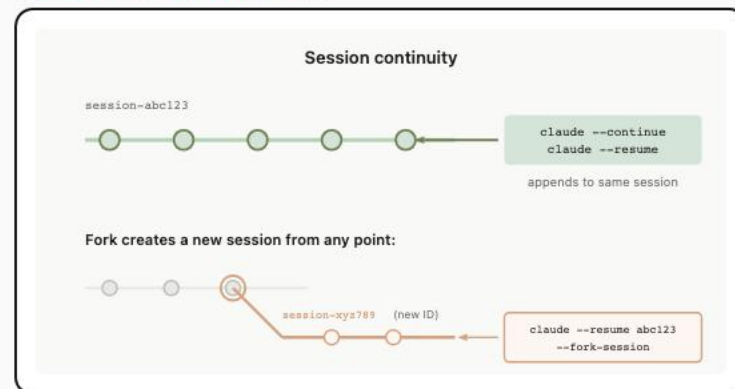
단순 작업은 비용만 늘 수 있음

multi-agent ~15x cost (Anthropic Whitepaper)

언제 **fresh context**로 회전하나

맥락이 흐려지면 더 버티지 말고 state를 남긴 뒤 새 context로 회전합니다 — Claude Code도 resume와 fork를 분리합니다.

CLAUDE CODE — RESUME vs FORK



code.claude.com/docs/ko/how-claude-code-works

맥락이 흐려지면

compact state →
fresh context →
continue

중요한 건 fresh context가 아니라 **handoff state**. State가 좋으면 새 agent가 바로 이어 받습니다.

무엇을 **memory/skill**로 승격하나

안정된 사실, 반복 교정, 재사용 가능한 절차만 장기화합니다. 임시 추론은 남기지 않습니다.

observation	stable fact	procedure	skill
--------------------	--------------------	------------------	--------------

위험과 가드레일

Loop는 강력합니다. 그래서 stop condition, verifier, git boundary, human gate가 harness의 일부입니다 — Claude Code는 이것 hook으로 강제합니다.

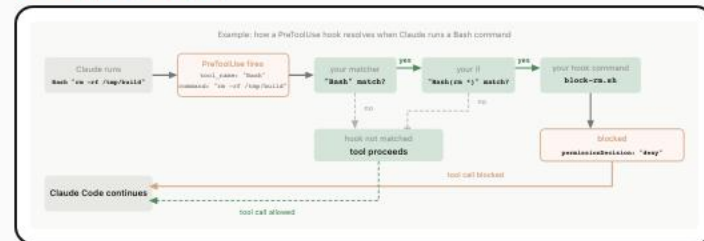
max iterations

tests

git branch

human gate

HOOK 해석 흐름 — 공식 도식



PRETOOLUSE — BASH RM 차단 (settings.json)

```

{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "if": "Bash(rm *)",
        "command":
          ".claude/hooks/block-rm.sh"
      }]
    }]
  }
}

```

매번 파일 편집 후 ESLint, 워킹 명령 차단 — code.claude.com/docs/en/hooks

프롬프트는 입력, harness는 반복의 구조

좋은 harness는 Claude가 실패하지 않게 만드는 장치가 아니라, 실패 후 더 나은 시작점을 만드는 장치입니다.

여러분의 **harness**는 여러분의 **loop** 모양이 어야 합니다

같은 네 축, 서로 다른 점유. 오늘은 자기 작업의 반복 구조를 설계하는 렌즈를 가져가면 됩니다.

Inject

Compact

Isolate

Persist

15분

여러분의 loop를 그려봅시다

어디서부터 시작할지, 어떤 loop로 만들지, 무엇을 verifier로 둘지 함께 지명해봅시다.